

CodeForge AI v2.0 - Enhanced Features Documentation

Overview

CodeForge AI v2.0 introduces five major enhancements to the platform:

1. **Local LLM Hosting** - Run AI models locally using HuggingFace Transformers
 2. **Default System Prompt Configuration** - Pre-configured prompts including DIA SAP Operator
 3. **GitHub Actions/Workflows Generator** - Automated CI/CD pipeline generation
 4. **Chat Parser & Code Extractor** - Extract code from AI chat transcripts
 5. **Code Completion & Interactive Development** - Intelligent code analysis and completion
-

1. Local LLM Hosting

Description

Host and run large language models locally using HuggingFace Transformers with support for Dolphin-Mistral-24B-Venice-Edition and other models.

Features

- Download models from HuggingFace Hub
- Load/unload models dynamically
- Support for 4-bit and 8-bit quantization
- CUDA/CPU/Auto device selection
- Streaming text generation
- Database persistence for configurations

API Endpoints

Get LLM Status

```
GET /api/local-llm/status
```

Response:

```
{
  "is_loaded": false,
  "cuda_available": true,
  "cuda_device_count": 1,
  "model_id": null
}
```

Download Model

```
POST /api/local-llm/download?model_id=dphn/Dolphin-Mistral-24B-Venice-Edition
```

Load Model

```
POST /api/local-llm/load
Content-Type: application/json
```

```
{
  "name": "Dolphin-Mistral",
  "model_id": "dphn/Dolphin-Mistral-24B-Venice-Edition",
  "device": "auto",
  "quantization": "4bit",
  "max_context_length": 8192
}
```

Generate Text

```
POST /api/local-llm/generate
Content-Type: application/json
```

```
{
  "prompt": "Write a Python function to calculate fibonacci",
  "system_prompt": "You are a helpful coding assistant",
  "temperature": 0.7,
  "max_tokens": 2048
}
```

Stream Generate

```
POST /api/local-llm/stream
Content-Type: application/json
```

```
{
  "prompt": "Explain async/await in JavaScript",
  "temperature": 0.7
}
```

Returns: Server-Sent Events stream

Default Model

The default model is **Dolphin-Mistral-24B-Venice-Edition** (dphn/Dolphin-Mistral-24B-Venice-Edition), optimized for code generation and instruction following.

2. Default System Prompt Configuration

Description

Pre-configured system prompts for various use cases, with database persistence and easy selection via API.

Available System Prompts

DIA SAP Operator

Specialized prompt for security research and development with:

- Deep technical knowledge of x86/x64 assembly, ARM
- Exploit development expertise
- Operating system internals
- Security-focused programming guidance

Other Prompts

- **Code Generation Expert** - Clean, efficient code generation
- **Code Review Assistant** - Comprehensive code reviews
- **Documentation Writer** - Technical documentation
- **DevOps Engineer** - CI/CD and infrastructure
- **Full-Stack Developer** - Web development guidance

API Endpoints

List System Prompts

```
GET /api/system-prompts
```

Get Specific Prompt

```
GET /api/system-prompts/{prompt id}
```

Use in Chat

```
POST /api/unified-chat
Content-Type: application/json
```

```
{
  "messages": [{"role": "user", "content": "Analyze this code for vulnerabilities"}],
  "provider": "venice",
  "system prompt id": "dia-sap-operator-id"
}
```

3. GitHub Actions/Workflows Generator

Description

Automatically generate GitHub Actions workflows for CI/CD pipelines based on project configuration.

Features

- Multi-language support (Python, JavaScript, TypeScript, Go, Rust, Java)
- Linting, testing, security scanning, and deployment pipelines
- Built-in templates for common workflows
- Custom template creation
- Dependabot configuration

API Endpoints

Generate Workflows

```
POST /api/workflows/generate
Content-Type: application/json

{
  "name": "My Project",
  "languages": ["python", "typescript"],
  "include linting": true,
  "include testing": true,
  "include security": true,
  "include build": true,
  "include deploy": false,
  "python version": "3.11",
  "node version": "18"
}
```

Response:

```
{
  "success": true,
  "workflows": [
    ".github/workflows/ci.yml",
    ".github/workflows/security.yml",
    ".github/dependabot.yml"
  ],
  "languages_detected": ["python", "typescript"]
}
```

Generate for Project

```
POST /api/workflows/generate-for-project/{project id}
Content-Type: application/json

{
  "name": "Project CI",
  "languages": ["python"],
  "include deploy": true,
  "deploy target": "docker"
}
```

List Templates

```
GET /api/workflows/templates?category=testing
```

Generated Files

The generator creates:

- ci.yml - Main CI pipeline with linting and testing
 - security.yml - CodeQL analysis and dependency review
 - deploy.yml - Deployment to Vercel, Docker, or AWS
 - dependabot.yml - Dependency updates configuration
-

4. Chat Parser & Code Extractor

Description

Parse AI chat transcripts (DOCX, PDF, TXT, HTML, MD) to extract code blocks and reconstruct project structures.

Features

- Multi-format support (DOCX, PDF, TXT, HTML, Markdown)
- Automatic language detection
- File path extraction from comments
- Project structure reconstruction
- Auto-generated README
- ZIP download

Supported Chat Formats

- ChatGPT exports
- Claude conversations
- Gemini transcripts
- Venice.ai chats
- DeepSeek outputs
- Kimi transcripts

API Endpoints

Upload and Parse

```
POST /api/chat-parser/upload
Content-Type: multipart/form-data

file: [document file]
file type: docx (optional)
```

Response:

```
{
  "success": true,
  "extraction_id": "uuid",
  "files": {
    "main.py": "import os\n...",
    "utils/helpers.py": "def format_string()..."
  },
  "code_blocks_count": 15,
  "languages_detected": ["python", "javascript"],
  "source_format": "ChatGPT",
  "readme_content": "# Extracted Project\n..."
}
```

Download as ZIP

```
POST /api/chat-parser/download-zip
Content-Type: multipart/form-data

file: [document file]
```

Returns: `extracted_project.zip`

Create Project from Extraction

```
POST /api/chat-parser/create-project
Content-Type: application/json

{
  "extraction id": "uuid",
  "project name": "My Extracted Project"
}
```

List Extractions

```
GET /api/chat-parser/extractions
```

Language Detection

The parser automatically detects 20+ programming languages including:

- Python, JavaScript, TypeScript
- Go, Rust, Java, C/C++
- HTML, CSS, SQL
- Shell, YAML, JSON
- Ruby, PHP, Swift, Kotlin

5. Code Completion & Interactive Development

Description

Intelligent code analysis to identify missing functions, incomplete implementations, and provide AI-powered completions.

Features

- Missing function detection
- Stub implementation identification
- AI-powered code completion
- Interactive Development Wizard
- Project scaffolding generation

API Endpoints

Analyze Code

POST /api/code-completion/analyze

Content-Type: application/json

```
{
  "code": "def calculate_total(items):\n    pass\n\nresult = calculate_total(order_items)",
  "language": "python",
  "file_path": "utils.py"
}
```

Response:

```
{
  "missing_functions": [
    {
      "name": "calculate_total",
      "file_path": "utils.py",
      "line_number": 1,
      "signature": null,
      "context": "def calculate_total(items):\n    pass"
    }
  ],
  "suggestions": [
    {
      "code": "def calculate_total(items):\n    return sum(item.price for item in items)",
      "explanation": "AI-generated implementation for calculate_total",
      "confidence": 0.8
    }
  ],
  "analysis_summary": {
    "total_missing": 1,
    "total_suggestions": 1,
    "language": "python"
  }
}
```

Complete Function

POST /api/code-completion/complete-function

Content-Type: application/json

```
{
  "code": "class OrderService:\n    def init (self):\n        self.db = Database()\n\n    def get_order(self, order id):\n        pass",
  "function name": "get_order",
  "language": "python",
  "requirements": "Should fetch from database and handle not found case"
}
```

Development Wizard

Start Wizard Session

```
POST /api/wizard/start?project_id=optional-project-id
```

Response:

```
{
  "session_id": "uuid",
  "phase": "structure",
  "decisions_count": 0,
  "next_question": {
    "id": "project_type",
    "question": "What type of project are you building?",
    "options": ["Web Application", "API/Backend", "CLI Tool", "Library/Package"]
  }
}
```

Answer Question

```
POST /api/wizard/answer
Content-Type: application/json
```

```
{
  "session_id": "uuid",
  "question_id": "project_type",
  "answer": "Web Application"
}
```

Generate Scaffolding

```
POST /api/wizard/{session_id}/generate
```

Response:

```
{
  "success": true,
  "files": ["README.md", "src/main.py", "tests/test_main.py", "requirements.txt"],
  "total_files": 12,
  "decisions_applied": 8
}
```

Get Generated Files

```
GET /api/wizard/{session_id}/files
```

Unified Chat API

Description

Single endpoint supporting both Venice API and Local LLM with automatic system prompt resolution.

Endpoint

```
POST /api/unified-chat
Content-Type: application/json

{
  "messages": [
    { "role": "user", "content": "Write a REST API in FastAPI" }
  ],
  "provider": "venice", // or "local"
  "model": "dolphin-2.9.2-qwen2-72b",
  "system_prompt_id": "code-generation-expert",
  "temperature": 0.7,
  "max_tokens": 4096,
  "stream": true
}
```

Provider Options

- `venice` - Use Venice.ai API (default)
- `local` - Use locally loaded LLM

Database Models

New Tables

local_llm_configs

Stores local LLM configurations:

- Model ID, path, quantization settings
- Device preference (cuda/cpu/auto)
- Default generation parameters

chat_extractions

Stores chat parsing results:

- Original filename and type
- Extracted files and structure
- Languages detected
- Source format (ChatGPT, Claude, etc.)

workflow_templates

Stores workflow templates:

- Template name and category
- YAML content
- Template variables
- Supported languages

code_completion_sessions

Stores wizard sessions:

- Project association
- Decisions made
- Generated artifacts
- Completion progress

Frontend Components

ChatParser Component

React component for uploading and parsing chat transcripts:

- Drag-and-drop file upload
- Real-time extraction preview
- File tree navigation
- ZIP download
- Project creation

Usage

```
import ChatParser from '@components/ChatParser'

export default function ParserPage() {
  return <ChatParser />
}
```

Configuration

Environment Variables

```
# Backend
DATABASE_URL=postgresql://user:pass@localhost:5432/codeforge
VENICE_API_KEY=your-api-key

# Local LLM (optional)
MODELS_DIR=/path/to/models
DEFAULT_DEVICE=auto # cuda, cpu, auto

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Requirements

New Python dependencies:

```
# Local LLM
torch>=2.1.0
transformers>=4.36.0
accelerate>=0.25.0
bitsandbytes>=0.42.0

# Document Parsing
python-docx>=1.1.0
PyPDF2>=3.0.0
beautifulsoup4>=4.12.0

# YAML Processing
PyYAML>=6.0.1
```

Getting Started

1. Update Dependencies

```
cd backend
pip install -r requirements.txt
```

2. Run Database Migrations

```
alembic upgrade head
```

3. Seed Default Data

```
python -c "from app.database.seeds import run_all_seeds; import asyncio;
asyncio.run(run_all_seeds())"
```

4. Start Backend

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

5. Start Frontend

```
cd frontend
npm run dev
```

Examples

Extract Code from ChatGPT Export

```
import requests

with open('chatgpt_export.docx', 'rb') as f:
    response = requests.post(
        'http://localhost:8000/api/chat-parser/upload',
        files={'file': f}
    )

result = response.json()
print(f"Extracted {result['code_blocks_count']} code blocks")
print(f"Languages: {result['languages_detected']}")
```

Generate CI Workflow

```
import requests

response = requests.post(
    'http://localhost:8000/api/workflows/generate',
    json={
        'name': 'My Python Project',
        'languages': ['python'],
        'include_linting': True,
        'include_testing': True,
        'include_security': True
    }
)

workflows = response.json()['workflows']
print(f"Generated: {workflows}")
```

Use Local LLM

```
import requests

# Load model
requests.post(
    'http://localhost:8000/api/local-llm/load',
    json={
        'name': 'Dolphin',
        'model_id': 'dphn/Dolphin-Mistral-24B-Venice-Edition',
        'quantization': '4bit'
    }
)

# Generate
response = requests.post(
    'http://localhost:8000/api/local-llm/generate',
    json={
        'prompt': 'Write a Python class for managing a task queue',
        'temperature': 0.7
    }
)

print(response.json()['response'])
```

Changelog

v2.0.0

- Added Local LLM hosting with HuggingFace Transformers
- Added Chat Parser for code extraction
- Added GitHub Actions Workflow Generator
- Added Code Completion and Analysis
- Added Development Wizard
- Added DIA SAP Operator system prompt
- Added unified chat API
- Enhanced database models
- Updated frontend API client